

KernelGuard Phase 2 Project Report

OS (TCS 502) and DBMS (TCS 503) | Academic session 2026–27

Graphic Era (Deemed to be University), Dehradun

Project field	Submission detail
Team ID	OSDBMS-V-2026-T298
Team name	KernelGuard
Mentor	(Prof.)Dr.Mohammad Wazid
Source snapshot	Phase 2 standalone project

Team member	Student ID	Area presented
Arshpreet Singh	24021835	Linux audit and host collection
Mohd Ahmed Khan	24021365	Database and persistence
Ankit	240212134	Detection rules and evaluation
Mohd Shoaib	240211124	Dashboard and review

Project Abstract

KernelGuard Phase 2 demonstrates a complete path from a Linux audit record to an explainable alert. The collector groups audit lines, extracts selected identity and file fields, and writes them to a dedicated MySQL database. Two policies identify repeated failed logins and successful access to a protected path outside configured hours. A read-only dashboard lets the examiner follow an alert back to its supporting events and saved rule settings. The standalone Phase 2 folder is a presentation copy reconstructed from the project, not a dated snapshot of the original submission.

Problem and Objectives

On a Linux host, access records can be difficult to connect with the policy that produced a warning. This phase tests whether a small relational model can preserve enough context for a reviewer to trace a controlled action from audit output through storage to an alert.

- Collect grouped Linux Audit records without treating a partial line as a complete event.
- Store event identity, audit details, policy settings and alert evidence in MySQL.
- Demonstrate two deterministic rules and a read-only review screen.
- Separate synthetic demonstration records from live operating-system records.

Updated Project Approach and Architecture

The Linux rule watches a protected directory. The Python collector polls ausearch, normalizes related records and marks live rows with origin=live. A unique source identifier suppresses replay. A MySQL advisory lock coordinates the writer, while event inserts, alert creation and the collector checkpoint share one transaction. The dashboard uses a SELECT-only database account. The optional nine-event fixture uses origin=demo and gives a repeatable classroom demonstration. The design follows documented audit fields and relational transaction behavior [1–3].

Stage	Implementation evidence
Audit input	linux/kernelguard.rules and kernelguard/parser.py
Persistence	Eight MySQL tables in kernelguard/core.py and sql/schema.mysql.sql
Detection	Failed-login threshold and outside-hours protected-file access
Review	Flask overview, filters, alert detail and evidence links

Tasks Completed

The standalone project includes the audit rule, parser, database model, two policies, read-only dashboard, SQL analysis queries, fixture, configuration validation and tests. The table records assigned presentation areas. It does not claim that the repository proves each person authored every listed file.

Member	Presentation responsibility
Arshpreet Singh	Linux audit and host collection
Mohd Ahmed Khan	Database and persistence
Ankit	Detection rules and evaluation
Mohd Shoaib	Dashboard and review

Testing and Validation Status

The fixture contains nine synthetic events. The documented demonstration yields two alerts and six alert-to-event links. A Kali host check dated 2 October 2026 reports a controlled protected-file read ingested as a live event, a working dashboard, and a passing four-test MySQL 8.4 integration run. These are recorded project checks, not measurements collected from this Windows copy today.

Check	Recorded outcome	Boundary
Synthetic fixture	9 events, 2 alerts, 6 evidence links	Repeatable demo, not live collection
Live protected read	Observed on Kali, 2 Oct 2026	One configured host and protected path
MySQL integration	4 tests passed on MySQL 8.4	Disposable test database
Dashboard	Read-only account and active collector reported	Localhost deployment

Challenges and Remaining Work

Linux Audit emits several lines for one event; grouping by audit identity is essential before a rule uses it. A restart can replay output, so the schema uses unique source IDs and a transactional checkpoint. Synthetic records are useful for a short evaluation but cannot prove live collection. Phase 3 extends the narrow prototype with user, process and USB inventory, further policies, authentication, review history and broader acceptance tests.

Codebase Information and Deliverables

Source folder: C:\KernalGuard-Phase2-PBL. Key files are app.py, kernelguard/core.py, kernelguard/parser.py, kernelguard/locking.py, linux/kernelguard.rules, sql/schema.mysql.sql, sql/analysis.sql, fixtures/demo.audit and tests/test_app.py. The folder is a standalone copy. No public repository URL or trustworthy individual commit count is present in the source, so those fields should be completed from the team's actual Git history before submission.

Demonstration Procedure

For a repeatable classroom run, initialize the dedicated MySQL database and import the labelled fixture with `python app.py demo`. Open the dashboard at the project's localhost port and first show the event count. Open each alert to show its evidence rows and the saved policy values. Re-running the fixture should not create duplicate source events. This demonstrates database relationships and deterministic rules, while the `origin=demo` label makes clear that the rows came from prepared input.

For a live Linux check, install the supplied rule on the configured protected directory and confirm it appears in `auditctl`. Start the collector and dashboard with the same database URL. Read a harmless protected test file as a normal lab user, inspect `ausearch` for the keyed audit event, then locate the matching `origin=live` row and any relevant alert in the dashboard. An outside-hours policy demonstration should use a lab configuration whose allowed hours exclude the current hour. The system clock should not be changed.

The event record and the alert-to-event link are the main evidence. A screenshot of a dashboard count alone cannot establish that a real audit event passed through the entire chain. Keep the audit serial, event origin, path, identity fields, rule setting and database row together in the evaluation record.

Database Learning Outcome

The eight-table model makes a clear distinction between one event, its audit-specific fields, a policy definition and an alert episode. A many-to-many join table allows an alert to cite several authentication attempts without duplicating the alert row. Foreign keys and unique source IDs protect relationships and replay identity. The collector's cursor advances in the same transaction as the newly accepted events, so a write failure does not silently skip input. The supplied SQL analysis file demonstrates grouping, joins and evidence inspection.

Originality and Scope

This report was written from the project implementation and recorded checks in fresh prose. It cites the underlying technologies and does not claim a plagiarism percentage. The system flags configured policy matches; it does not establish intent, stop access or measure field detection accuracy. The team should disclose AI assistance according to university policy and verify that personal contribution claims match its own work history.

References

[1] Red Hat, Understanding Audit Log Files,
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/security_guide/sec-understanding_audit_log_files

[2] SQLAlchemy 2.0, Working with Engines and Connections,
<https://docs.sqlalchemy.org/en/20/core/connections.html>

[3] MySQL 8.4 Reference Manual, FOREIGN KEY Constraints,
<https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html>

[4] KernelGuard Phase 2 source, README.md, docs/TEAM.md and tests/test_app.py in C:\KernalGuard-Phase2-PBL.